

CENTRO UNIVERSITÁRIO FUNDAÇÃO SANTO ANDRÉ

Gabriel Ávila de Oliveira

Nicholas Vincent Perin

Vitor Borges Souza

GRUPO 2 – Eficiência Energética

**Projeto Firmware para Máquina de Lavar Doméstica com
ESP32**

SANTO ANDRÉ

2026

Gabriel Ávila de Oliveira

Nicholas Vincent Perin

Vitor Borges Souza

GRUPO 2 – Eficiência Energética

**Projeto Firmware para Máquina de Lavar Doméstica com
ESP32**

Trabalho apresentado como requisito parcial para obtenção do título de Bacharel de Engenharia da Computação pelo Centro Universitário Fundação Santo André.

Orientador: Prof. Edson Antônio De Abreu

SANTO ANDRÉ

2026

SUMÁRIO

SUMÁRIO.....	3
1 INTRODUÇÃO.....	4
2 HARDWARE.....	5
3 FLUXOGRAMA	7
4 FUNÇÕES / BIBLIOTECAS	8
4.1 BIBLIOTECAS INTERNAS	8
4.2 BIBLIOTECAS EXTERNAS.....	8
4.3 TIMERS	8
4.4 PWM	9
5 CONCLUSÃO	11
6 BIBLIOGRAFIA.....	12

1 INTRODUÇÃO

Este projeto apresenta o firmware desenvolvido para uma simulação de máquina de lavar inteligente, implementado em um microcontrolador ESP32 como parte da avaliação P2 da disciplina de Microcontroladores.

O principal objetivo do sistema é demonstrar o controle eficiente de um processo sequencial por meio de uma máquina de estados, cobrindo as etapas típicas de um ciclo de lavagem: enchimento, lavagem, escoamento, enxágue, segundo escoamento e centrifugação. Cada etapa é temporizada por hardware timers dedicados do ESP32.

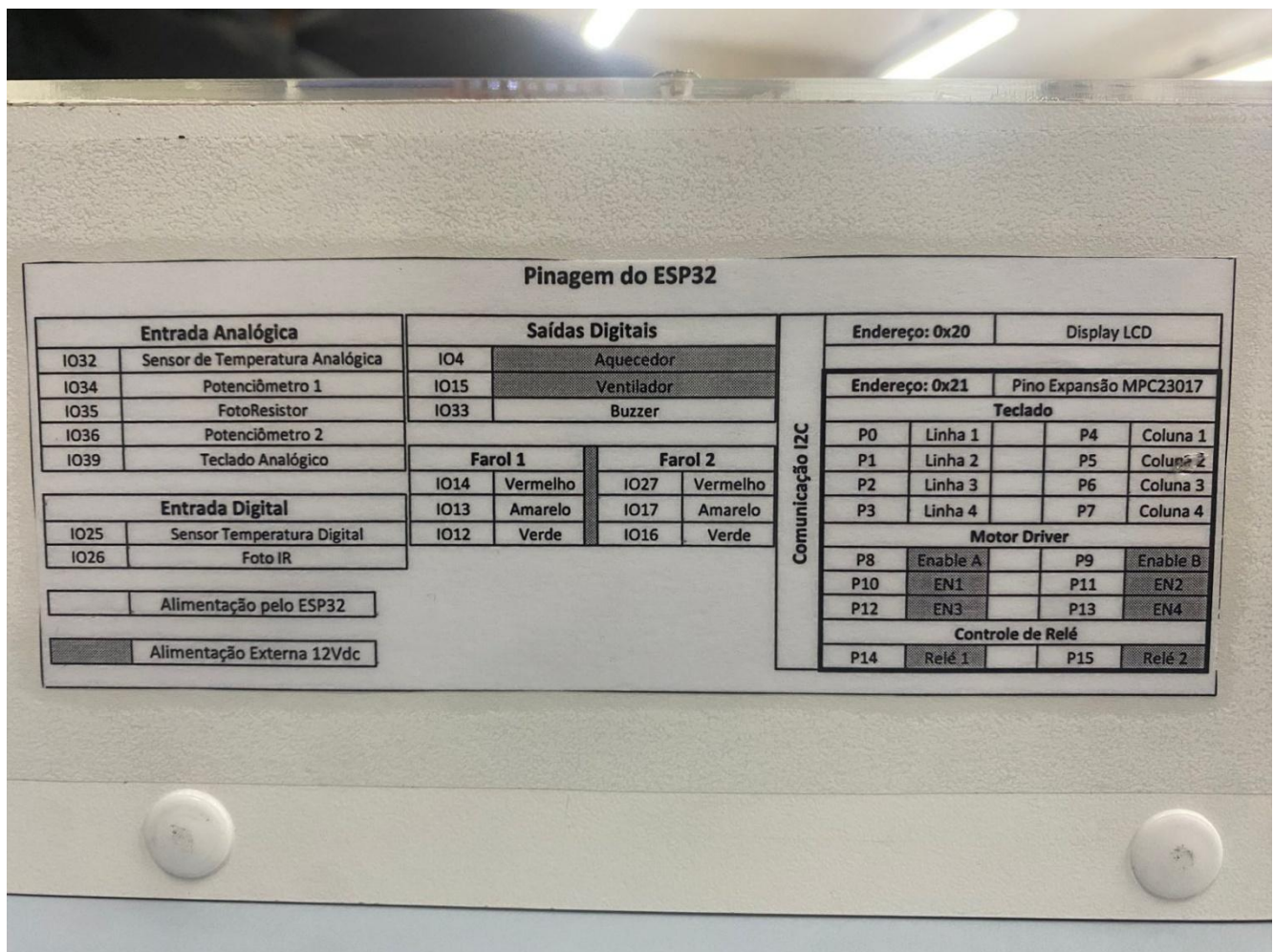
O foco central do trabalho é a eficiência energética via controle PWM do motor. O duty cycle não é fixo: ele é calculado dinamicamente com base no nível de água capturado ao final do enchimento (valor "congelado" para evitar variações durante o ciclo). Isso significa que uma máquina com pouca roupa gira com menos potência.

Para interação com o usuário, o sistema conta com duas interfaces de entrada: um teclado 4×4 conectado via expensor de I/O MCP23017 (protocolo I²C), e comunicação Bluetooth Serial, permitindo controle remoto pelo celular Android. O teclado tem prioridade, mas o Bluetooth entra em ação quando nenhuma tecla física é pressionada.

Como saída, o sistema utiliza um display LCD 16×2 via I²C para exibição do estado atual, potência, eficiência e tempo restante da etapa. Um buzzer emite sinais sonoros nas transições de etapa e ao final do ciclo.

Durante os processos, todas as informações de estado e de eficiência energética são reportadas via serial, permitindo também o monitoramento via celular. Todo o consumo de energia é acumulado em tempo real (em Wh) e reportado via Bluetooth e LCD ao término do ciclo.

2 HARDWARE

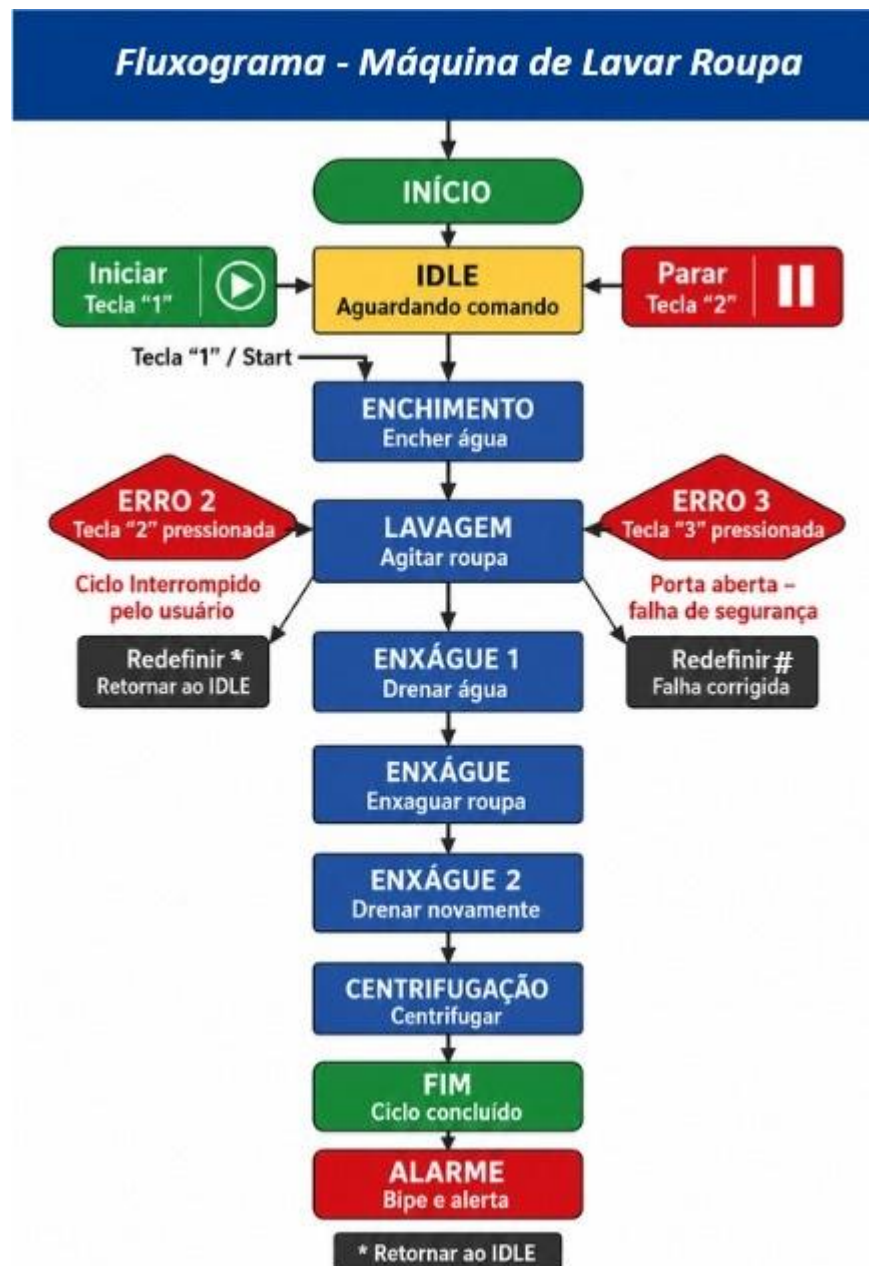


Fazemos uso do módulo oferecido pela FSA, do qual possui uma placa ESP32 ligado a potenciômetros, faróis de LED, um buzzer, um display LCD, e um teclado 4x4 conectado via expansão. Todos os pontos IO e endereços I2C estão destacados na imagem acima. Sobre cada um e seu uso no código:

1. Usamos o ponto IO36 (potenciômetro) para leitura do nível de água dentro da máquina de lavar, do qual pode ser ajustado na etapa de enchimento. Este valor é lido pelo código e então convertido para um valor entre 0% a 100%. Após passado o tempo de enchimento, o valor é congelado e usado para o restante dos cálculos do PWM.
2. Os pontos IO14, IO13, IO12, IO27, IO17 e IO16 são para os faróis, que indicam visualmente qual etapa do processo está rodando.

3. O ponto IO33 é o do buzzer, que produz um efeito sonoro ao avançar a cada etapa do processo, além de notificar quando tudo foi finalizado e o resultado do cálculo da eficiência energética está no display LCD.
4. O endereço I2C 0x20 é o da tela LCD, do qual exibe informações sobre cada etapa do processo de lavagem: nível de água, potência do motor, eficiência energética, entre outros.
5. O endereço I2C 0x21 é sobre os pinos de expansão do ESP32. Neste caso, ela possui um teclado externo 4x4. Dentro do código, há uma função lerTeclado() que consegue mapear os inputs recebidos do teclado e associar a um comando.

3 FLUXOGRAMA



Acima, segue o nosso fluxograma desenvolvido para o projeto. Este foi feito antes de todo o código, para podermos ter uma base sólida sobre o que implementar ao decorrer do tempo. Seguimos implementando as funções de cima para baixo de acordo com o fluxograma, até o final do código.

4 FUNÇÕES / BIBLIOTECAS

4.1 Bibliotecas Internas

O código faz uso de 3 bibliotecas internas:

- <Arduino.h>, a biblioteca padrão de qualquer código Arduino. Permite a execução de comandos básicos para funcionamento.
- <Wire.h>, a biblioteca que permite a comunicação I2C entre os periféricos e o ESP32. Neste caso, estamos usando ela para se receber os bits do display LCD e os pinos de expansão instalados.
- <BluetoothSerial.h>, a biblioteca que permite a comunicação Bluetooth com aparelhos próximos. Com essa biblioteca, podemos dar comandos com o celular para operar e monitorar a nossa máquina de lavar.

Todas estas bibliotecas já estão inclusas com o módulo ESP32, portanto podemos apenas incluir elas sem precisar instalar por fora.

4.2 Bibliotecas Externas

O código exige apenas uma biblioteca externa:

- <LiquidCrystal_I2C.h>, a biblioteca que simplifica a comunicação do nosso código com o display LCD. Ao invés de toda vez que precisar usar o LCD, interpretar os bits para identificar qual tecla foi pressionada, podemos fazer a interpretação uma vez com o Wire.h, e depois a LiquidCrystal.h simplifica com suas funções internas, dos quais são mais práticos e visualmente fáceis de interpretar.

4.3 Timers

O código faz uso de 3 timers:

- timerMS, do qual substitui millis() e delay(). Frequência base de 1MHz, alarme a cada 1.000 ticks = 1ms. ISR incrementa contadorMS, lido por getMS() e esperarMS(). Ativo o tempo todo.

- timerCiclo controla o tempo de cada etapa da máquina. Frequência base de 1MHz, alarme a cada 1.000.000 ticks = 1s. ISR incrementa contagemSegundos e seta timerDisparou quando a etapa termina. Ligado/desligado conforme a etapa.
- timerPWM gera o PWM do motor por software. Frequência base de 100kHz, alarme a cada 10 ticks = 100µs. ISR cicla um contador de 0–99 e compara com dutyCycleMotor para definir se o motor está ligado ou desligado no tick atual, resultando em um ciclo PWM de 10ms (100Hz). Ativo o tempo todo.

4.4 PWM

O firmware implementa um controle de largura de pulso (PWM), utilizando o timerPWM como base de tempo. O timer é configurado com frequência base de 100 kHz e alarme a cada 10 ticks, resultando em uma ISR disparada a cada 100 µs. Internamente, a ISR mantém um contador cíclico contadorPWM entre 0 e 99, de forma que o período completo de um ciclo PWM corresponde a $100 \times 100 \mu s = 10 \text{ ms}$, equivalente a uma frequência de 100 Hz.

A cada disparo da ISR, o estado do motor é definido pela comparação entre contadorPWM e o valor de dutyCycleMotor: se contadorPWM for menor que dutyCycleMotor, o motor é mantido ativo; caso contrário, é desligado. Dessa forma, um valor de dutyCycleMotor igual a D corresponde a uma razão cíclica efetiva de $d = D / 100$, representando a fração do período em que o motor permanece energizado.

O duty cycle de cada etapa é determinado em função do nível de água registrado no enchimento (nivelCongelado, N), mapeado linearmente: na lavagem, $D = 30 + 0,7 \times N$; no enxague, $D = 20 + 0,4 \times N$; na centrifugação, D é fixo em 95%. A potência instantânea entregue ao motor é modelada como proporcional ao quadrado da razão cíclica, $P = 500 \times d^2$, comportamento típico de cargas controladas por PWM, e a corrente estimada é obtida por $I = P / 220$. O índice de eficiência energética é calculado como $\eta = 100 - (d \times 100 \times (1 - n))$, onde $n = N / 100$ é o nível normalizado, expressando que a operação é mais eficiente quanto maior o nível de água e menor o esforço do motor. Por fim, a energia total consumida é acumulada a cada iteração do loop com passo de 10 ms, convertida para watt-hora pelo fator 1/3600.

Todos os cálculos em todas as etapas são registradas no monitor serial, permitindo monitoramento via celular, e também exibidas em tempo real na tela LCD.

5 CONCLUSÃO

O projeto abrange diversos conceitos, sendo que é um código para simular uma máquina de lavar: implementação de timers ao invés das funções `delay()` e `millis()`, implementação de PWM para controle de potência do motor e controle via Bluetooth Serial. Sem dúvidas, de longe o projeto não aparenta ser complexo, porém ter que balancear tudo em C++ para garantir que tudo compile sem problemas, além de implementar nosso objetivo específico, mostrou que coisas simples como uma máquina de lavar roupas tem toda uma engenharia complexa por trás.

Sobre as dificuldades enfrentadas, percebemos que organizar todo o cálculo e controle do PWM em forma de código se provou bem complexo. Também, pensamos em apenas usar bibliotecas que já vem com o Arduino, porém o código se tornou extremamente longo e instável com o controle do teclado externo sendo feito apenas com a biblioteca `<Wire.h>`. Por este motivo, utilizamos também a biblioteca `<LiquidCrystal_I2C.h>`.

Mas o maior problema de todos provou ser a troca da função `delay()` e `millis()` por timers via registradores. As outras duas funções podem ser implementadas simplesmente com uma linha de código, mas o uso de timers estendeu o nosso projeto em muitas linhas adicionais. Balancear não só a quantidade de timers, mas o quanto cada um tem de tempo demorou, mas conseguimos uma base sólida ao decorrer do projeto.

Todo o projeto em si foi uma experiência única, juntando os componentes físicos do módulo Arduino da FSA com o nosso código. Foi extremamente satisfatório ver o nosso projeto rodar de uma vez, exatamente como planejamos.

6 BIBLIOGRAFIA

Makiyama, M. (12 de Março de 2025). *VicorVision*. Fonte: Entenda o que é PWM, para que serve, como funciona e suas aplicações: <https://victorvision.com.br/blog/o-que-e-pwm/>

Pedro. (24 de Janeiro de 2026). *Clube do Maker*. Fonte: O que é PWM (Pulse-Width-Modulation)? Funcionamento e aplicações: <https://clubedomaker.com/o-que-e-pwm-pulse-width-modulation>

Souza, F. (10 de Janeiro de 2014). *Embarcados*. Fonte: PWM de Arduino: <https://embarcados.com.br/pwm-do-arduino/>